



[[[JAI SRI GURUDEV]]]
Sri AdichunchanagiriShikshana Trust ®

SJB INSTITUTE OF TECHNOLOGY

(A Constituent Institution of BGS & SJBIT Group of Institution & Hospitals)
No. 67, BGS Health & Education City, Dr. Vishnuvardhan Road, Kengeri, Bengaluru - 560 060



Department of Computer Science & Engineering



Subject: Theory of Computation
Subject Code: 23CSI602

Module 1: Introduction

Prepared By: Dr. Veena H N

FINITE AUTOMATA

Finite automata are basic modules of a computer system, which addresses simple real time problems. Figure 1.1 shows conceptual view of finite automata. It has an input tape, read head and finite controller. Input tape is divided into finite cells and each cell stores one input symbols. Read head is used to read the input symbol from the input tape. The finite controller used to record the next state of finite automata.

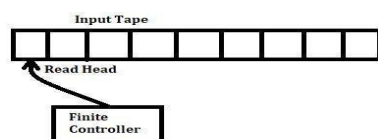


Figure 1.1: Conceptual view of finite automata

Before discussing finite automata in detail, it is very much important to know the basic requirement for the finite automata. They are

- i. Sets.
- ii. Graphs.
- iii. Languages.
- iv. Starclosure.
- v. Plusclosure.

Sets (S):

A set is a collection of elements, each element has its own identity. A set can be represented by enclosing its elements in curly braces. For example, the set of small letters a, b, c is shown as

$$S = \{a, b, c\}$$

Set $\{2, 4, 6, \dots\}$ denotes the set of all positive even integers. We can use more explicit notation, in which we write

$$S = \{i \mid i \text{ is } > 0 \text{ and is even}\}$$

We can perform following operations on sets, they are

- a. union ($\cup$),
- b. intersection ($\cap$), and
- c. difference ($-$)
- d. complementation defined as

$$S_1 \cup S_2 = \{x \mid x \in S_1 \text{ or } x \in S_2\}$$

$$S_1 \cap S_2 = \{x \mid x \in S_1 \text{ and } x \in S_2\}$$

$$S_1 - S_2 = \{x \mid x \in S_1 \text{ and } x \notin S_2\}$$

The complement of a set S , denoted by $\bar{S}$ consists of all elements not in S . To make this meaningful, we need to know what the universal set U of all possible elements is. If U is specified, then

$$\bar{S} = \{x \mid x \in U, x \notin S\}$$

This set with no elements, called the empty set or the null set, is denoted by ϵ . From the definition of a set, it is obvious that

$$S \cup \varepsilon = S \quad S - \varepsilon = S$$

$$S \cap \varepsilon = \varepsilon \quad \bar{\varepsilon} =$$

$$U$$

DeMorgan's laws: The following useful identities, known as DeMorgan's laws.

$$\overline{S_1 \cup S_2} = \bar{S}_1 \cap \bar{S}_2$$

$$\overline{S_1 \cap S_2} = \bar{S}_1 \cup \bar{S}_2$$

are needed on several occasions.

Subset and Proper Subset: A set S_1 is said to be a subset of S , every element of S_1 is also an element of S . We write this as $S_1 \subseteq S$. If $S_1 \subseteq S$, but S contains an element not in S_1 , we say that S_1 is a proper subset of S ; we write this as $S_1 \subset S$.

Disjoint Sets: If S_1 and S_2 have no common element, that is, $S_1 \cap S_2 = \varepsilon$ then the sets are said to be disjoint.

Finite and Infinite sets: A set is said to be finite if it contains a finite number of elements; otherwise it is infinite. The size of a finite set is the number of elements in it; this is denoted by $|S|$.

Power-Set: A given set has many subsets. The set of all subsets is called the power-set and is denoted by 2^S . Observe that 2^S is a set of sets.

If S is the set $\{a, b, c\}$, then its power set is

$$2^S = \{\varepsilon, \{a\}, \{b\}, \{c\}, \{a, b\}, \{b, c\}, \{a, c\}, \{a, b, c\}\} \text{ Here } |S| = 3$$

$$\text{and } |2^S| = 8.$$

Cartesian product: Ordered sequences of elements from other sets are said to be the Cartesian product. For the Cartesian product of two sets, which itself is a set of ordered pairs, we write

$$S = S_1 \times S_2$$

Let $S_1 = \{2, 4\}$ and $S_2 = \{2, 3, 5, 6\}$. Then $S_1 \times S_2 = \{(2, 2), (2, 3), (2, 5), (2, 6), (4, 2), (4, 3), (4, 5), (4, 6)\}$. Note that the order in which the elements of a pair are written matters. The pair $(4, 2)$ is in $S_1 \times S_2$, but $(2, 4)$ is not.

Graphs:

A graph is a construct consisting of two finite sets, the set $V = \{v_1, v_2, \dots, v_n\}$ of vertices and the set $E = \{e_1, e_2, \dots, e_m\}$ of edges. Each edge is a pair of vertices from V , for instance, $e_i = (v_i, v_j)$.

Languages:

A finite, nonempty set of symbols are called the alphabet and is represented with symbol Σ . From the individual symbols we construct strings, which are finite sequences of symbols from the alphabet. For example, if the alphabet $\Sigma = \{0\}$, then Σ^* to denote the set of strings obtained by concatenating zero or more symbols from Σ .

$$\text{We can write } \Sigma^* = \{\varepsilon, 0, 00, 000, \dots\}$$

If $\Sigma = \{0,1\}$

0 and 1 are input alphabets or symbols, then:

($0^2 = 00, 0^3 = 000, 1^2 = 11, 1^3 = 111$, whereas in mathematics they are numbers $0^2 = 0, 1^2 = 1$)

$L = \Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, \dots\}$

Kleenclosure(*closure):

$\Sigma^* = \{\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \dots\}$

$\Sigma^0 = \{\epsilon\}$ $\Sigma^1 = \{0,1\}$ $\Sigma^2 = \{00, 01, 10, 11\}$

$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

Therefore $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, \dots\}$

*Closure on $\Sigma = \{0,1\}$

$\Sigma^+ = \{\Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \dots\}$

$\Sigma^* = \Sigma^+ + \epsilon$

Therefore $\Sigma^+ = \Sigma^* - \epsilon$

Formal Languages and Automata Theory: It is a mathematical model used to design computer programs

and sequential logic circuits. It is state machines which capture and make transitions to accept or reject.

Finite automata has input tape, read head and finite controller. **Input tape:** It is divided into finite cells and each cell is used to store one input symbol. **Read head:** Read head is used to read input symbol from input tape.

Finite Controller: Finite controller records the transition to next state. The default value of finite controller is initial state q_0 . States are represented with circles; they are of three types,

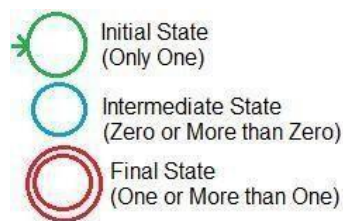


Figure 1.2: Different types of states used in finite automata

States are connected by lines called transitions. It has two ends, one end is called initiation point and the other end is called termination point ($>$).

Initiation $\longrightarrow$ Termination

The combination of transitions and states is called a transition diagram.



Figure 1.3: A typical transition diagram

Figure 1.4 shows a transition diagram to map cities and their roads.

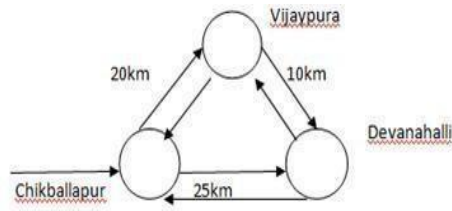


Figure 1.4: Finite automata with real time example.

Finite Automata are classified into two types. They are:

- 1) Deterministic Finite Automata (DFA)
- 2) Non Deterministic Finite Automata (NFA)

In Deterministic Finite Automata number of outgoing transitions are well defined from each state depending upon input symbols. For example, if there is one input symbol, then you find one outgoing transition from each state, for two input symbols two outgoing transitions are present and so on.

Example 1.1: Draw the DFA, the input symbols $\Sigma = \{0\}$

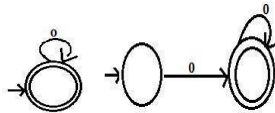


Figure 1.5: DFA's for input symbol $\Sigma = \{0\}$.

Example 1.2: Draw the DFA, the input symbols $\Sigma = \{0, 1\}$

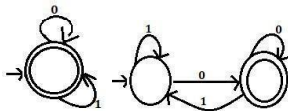


Figure 1.6: DFA's for input symbol $\Sigma = \{0, 1\}$.

Definition of DFA: we define DFA as,

$$M_{DFA} = (Q, \Sigma, \delta, \text{Initial state}, \text{Final state})$$

Where, Q is set of states, Σ = set of input symbols, δ : transition function is From the $Q \times \Sigma \rightarrow Q$ table 1.

$$Q = \{q_0, q_1, q_2\} \quad \Sigma = \{a, b, c\}$$

δ :

$$\delta(q_0, a) = q_1 \quad \delta(q_0, b) = q_0 \quad \delta(q_0, c) = q_2$$

$$\delta(q_1, a) = q_1 \quad \delta(q_1, b) = q_2 \quad \delta(q_1, c) = q_0$$

$$\delta(q_2, a) = q_2 \quad \delta(q_2, b) = q_1 \quad \delta(q_2, c) = q_2$$

Note: Initial State is q_0 and final state is q_2

Table 1.1: Transition table

$Q \backslash \Sigma$	a	b	c
$\rightarrow q_0$	q_1	q_0	q_2
q_1	q_1	q_2	q_0
$*q_2$	q_2	q_1	q_2

Nondeterministic Finite Automata (NFA):

If outgoing transitions are not dependent on number of input symbols, then such automata are called as Nondeterministic Finite Automata.

For example: $\Sigma = \{0, 1\}$

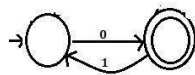


Figure 1.7: Nondeterministic Finite Automata for $\Sigma = \{0, 1\}$

There is no outgoing transition from initial state, for symbol '1' and similarly there is no outgoing transition from final state for '0'.

Transition Function for string:

Note: 1) δ is the transition function, used for symbols.

2) For the string (Set of symbols), $\hat{\delta}$ is used

$\{\} = \epsilon$

$\{a\} = a$

$\hat{\delta}(q_0, \epsilon) = q_0$ // q_0 is reading empty string, string is empty therefore, transition to same state q_0 .

We already Proved $\hat{\delta}(q_0, \epsilon) = q_0$

$\hat{\delta}(q_0, a) = \hat{\delta}(q_0, \epsilon a) = \delta(\hat{\delta}(q_0, \epsilon), a) = \delta(q_0, a)$ Therefore

$\hat{\delta}(q_0, a) = \delta(q_0, a)$

Therefore for single symbol string $\hat{\delta} = \delta$

$\hat{\delta}(q_0, wa)$

Where 'w' is the multi-symbol string and 'a' is a symbol, hence we separate the transition function for strings and symbols as given below:

$\delta(\hat{\delta}(q_0, w), a)$

Example1.3: Write a transition table for a DFA shown in figure 1.8. DFA over

alphabet $\Sigma = \{a, b\}$

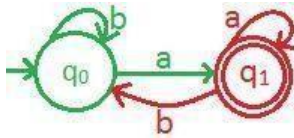


Figure 1.8: Transition diagram of a DFA.

Table 1.2: Transition table for the figure 1.8

$Q \backslash \Sigma$	a	b
$\rightarrow q_0$	$\delta(q_0, a)$	$\delta(q_0, b)$
$*q_1$	$\delta(q_1, a)$	$\delta(q_1, b)$

$Q \backslash \Sigma$	a	b
$\rightarrow q_0$	q_1	q_0
$*q_1$	q_1	q_0

$$M_{DFA} = (Q, \Sigma, \delta, IS, FS) \quad Q$$

$$= \{q_0, q_1\}$$

$$\Sigma = \{a, b\}$$

$$\delta(q_0, a) = q_1 \quad \delta(q_0, b) = q_0$$

$$= q_0 \delta(q_1, a) = q_1 \delta(q_1, b) = q_0$$

$$= q_0 \quad FS = \{q_1\}$$

Example1.4: Construct transition table for the diagram shown in figure 1.9, over the alphabet $\Sigma = \{0, 1, 2\}$

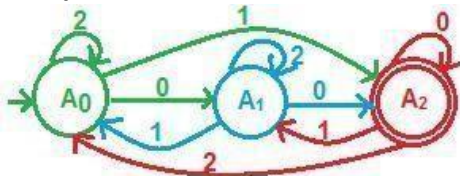


Figure 1.9: Transition diagram

Table 1.3: Transition table for the figure 1.9

$A \backslash \Sigma$	0	1	2
$\rightarrow A_0$	A_1	A_2	A_0
A_1	A_2	A_0	A_1
$*A_2$	A_2	A_1	A_0

We can define above DFA as: $M_{DFA} = (A, \Sigma, \delta, IS, \{FS\})$ δ :

$$\delta(A_0, 0) = A_1$$

$$\delta(A_0, 1) = A_2 \quad \delta(A_0, 2) = A_0$$

$$A_1, 0) = A_2$$

$$\delta(A_1, 1) = A_0 \quad \delta(A_1, 2) = A_1$$

$$\delta(A_2, 0) = A_2 \quad \delta(A_2, 1) = A_1 \quad \delta(A_2, 2) = A_0$$

$$A = \{A_0, A_1, A_2\}; \Sigma = \{0, 1, 2\}; IS = A_0; FS = A_2$$

String Acceptance:

The first symbol of string begins with initial state. Later, traverses zero or more than zero intermediate state(s) for the intermediate symbol(s). Finally, last symbol of the string ends with final state of automata. This is called string accepted or otherwise string is not accepted.

Example 1.5: Find out whether the given string is accepted by the DFA. Assume the string as $abab$. From the given string $\Sigma = \{a, b\}$. Note that always string is traversed by left side to right side,

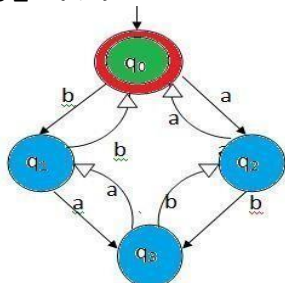


Figure 1.10: Transition diagram for example 1.5.

$$\delta(q_0, a) = \delta(q_0, a) = q_2 \quad \text{--- 1}$$

$$\delta(q_0, ab) = \delta(\delta(q_0, a), b) \text{ from 1 } \delta(q_0, a) = q_2$$

$$= \delta(q_2, b) = q_3 \text{--- 2}$$

$$\delta(q_0, aba) = \delta(\delta(q_0, ab), a) \text{ from equation 2 } \delta(q_0, ab) = q_3$$

$$= \delta(q_3, a) = q_1$$

$$\delta(q_0, abab) = \delta(\delta(q_0, aba), b)$$

$$= \delta(q_1, b) =$$

Find out whether given string 10110 is accepted by DFA.

From given string $\Sigma = \{0, 1\}$

10110

$$(q_0, 1) = \delta(q_0, 1) = q_2 \quad \text{--- ①}$$

10110

$$(q_0, 10) = \delta(q_0, 1), 0 = \delta(q_2, 0) = q_2 \quad \text{--- ②}$$

$$\text{10110} \quad (q_0, 101) = \delta(q_0, 10), 1 = \delta(q_2, 1) = q_1$$

$$\text{10110} \quad (q_0, 1011) = \delta(q_0, 101), 1 = \delta(q_1, 1) = q_2$$

$$\text{10110} \quad (q_0, 10110) = \delta(q_0, 1011), 0 = \delta(q_2, 0) = q_2$$

q_2 is a final state hence given string 10110 is accepted.

Given string is accepted by a DFA.

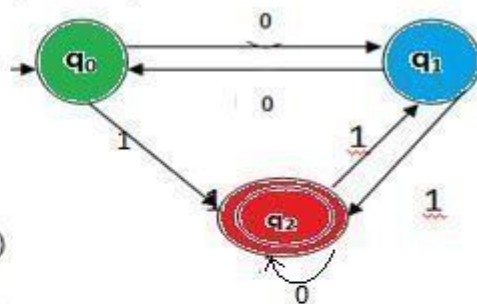


Figure 1.11 : Transition diagram

Example 1.6: Find out the language accepted by the DFA.

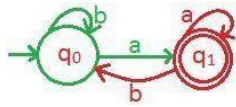


Figure 1.12: Transition diagram of Example 1.6.

$\delta(q_0, a) = q_1$; $\hat{\delta}(q_0, aa) = \delta(\hat{\delta}(q_0, a), a) = \delta(q_1, a) = q_1$
 $L = \{a, aa, ba, baa, aba, \dots\}$

In general we can write:

$L = \{w \mid w \text{ is a string ending with symbol } a\}$

Example 1.7: Find out the language accepted by the DFA.

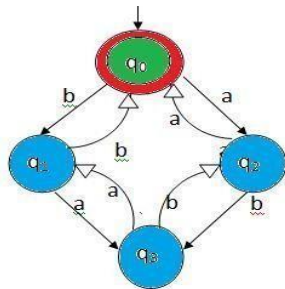


Figure 1.13: Transition diagram of Example 1.7

Initial state is also a final state, hence it accepts empty string ϵ .

$\delta(q_0, \epsilon) = q_0$;

$\hat{\delta}(q_0, aa) = \delta(\hat{\delta}(q_0, a), a) = \delta(q_2, a) = q_0$;

$\hat{\delta}(q_0, bb) = \delta(\hat{\delta}(q_0, b), b) = \delta(q_1, b) = q_0$;

Therefore language $L = \{\epsilon, aa, bb, aabb, abab, \dots\}$

In general we can write $L = \{w \mid w \text{ is a string consisting of even number of } a\text{'s and even number of } b\text{'s}\}$

Design of DFA:

Example 1.8: Design a DFA which accepts all the strings ending with symbol 'a' over the alphabet $\Sigma = \{a, b\}$.

$L = \{a, aa, ba, aba, bba, \dots\}$

- i. Draw the NFA for the smallest string of the language. The NFA for the string 'a' is

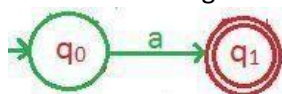


Figure 1.14: NFA for the string 'a'

Convert all states of NFA to DFA, that is from each state two outgoing transitions, one for symbol 'a' and another for 'b'. From initial state only one outgoing transition on symbol 'a', one more outgoing transition is required that is on 'b'. If the condition in the given problem ends with, the outgoing

transition on the other symbol b, definitely loop to initial state.

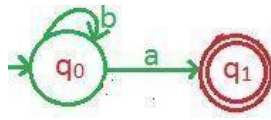


Figure 1.14: second outgoing transition from state q_0

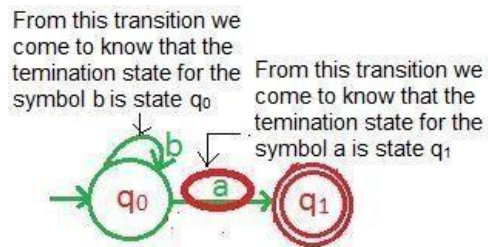


Figure 1.15: q_0 is termination for symbol b and q_1 is termination for symbol a

Termination state for symbol 'a' is q_1 and for 'b' it is q_0 . Outgoing transition from q_1 on symbol 'a' is loop, and for 'b' it is to q_0 .

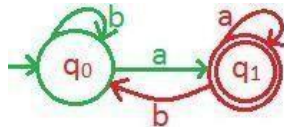


Figure 1.16: DFA which accept all the strings ends with a. We can

define above DFA as $M_{DFA} = (Q, \Sigma, \delta, IS, \{FS\})$

$Q = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

$\delta(q_0, a) = q_1, \delta(q_0, b) = q_0$

$\delta(q_1, a) = q_1, \delta(q_1, b) = q_0$

$IS = \{q_0\}$

$FS = \{q_1\}$

Example 1.8: Design a DFA which accept all the strings ends with substring 01 over the alphabet

$\Sigma = \{0, 1\}$.

$L = \{01, 001, 101, 0001, 1001, \dots\}$.

- i. Draw the NFA for the smallest string 01 of the language. The NFA for the string 01 is

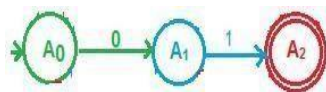


Figure 1.17: NFA for the string 01.

- ii. Convert all states of NFA to DFA, that is from each state two outgoing transitions, one for symbol '0' another for '1'. From initial state only one outgoing transition on symbol '0', one

more outgoing transition is required that is on '1'. If the condition in the given problem is, ends with, the outgoing transition on the other symbol 1, definitely loops to a initial state.

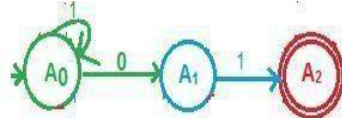


Figure 1.18: conversion of NFA state A_0 to DFA

From this transition we come to know that the termination state for the symbol '1' is state A_0

From this transition we come to know that the termination state for the symbol '0' is state A_1

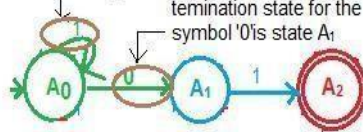


Figure 1.19: A_0 is termination for symbol 1 and A_1 is termination for symbol 0

Termination state for symbol '0' is A_1 and for '1' it is A_0 . Outgoing transition from A_1 on symbol '0' is loop. From A_2 on symbol '0' to A_1 and on symbol '1' to A_0 .

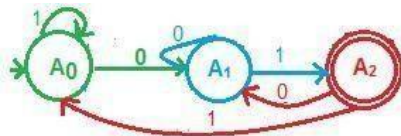


Figure 1.20: DFA which accepts all strings ending with 01. We

can define above DFA as $M_{DFA} = (A, \Sigma, \delta, IS, \{FS\})$

$A = \{A_0, A_1, A_2\}; \Sigma = \{0, 1\}$

δ :

$\delta(A_0, 0) = A_1, \delta(A_0, 1) = A_0$

$\delta(A_1, 0) = A_1, \delta(A_1, 1) = A_2$

$\delta(A_2, 0) = A_1, \delta(A_2, 1) = A_0$

$FS = A_2$

Example 1.9: Design a DFA which accepts all strings ending with substring 012 over the alphabet

$\Sigma = \{0, 1, 2\}$.

$L = \{012, 0012, 1012, 00012, 10012, \dots\}$.

- Draw the NFA for the smallest string 012 of the language. The NFA for the string 012 is

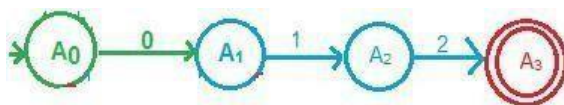


Figure 1.21: NFA for the string 012.

- Convert all states of NFA to DFA, that is from each state three outgoing transitions, one for symbol '0' second for '1' third from '2'. From initial state only one outgoing transition on symbol '0', two more outgoing transitions are required that is on '1' and '2'. If the condition

in the given problem ends with, the outgoing transition on the other symbols '1' and '2', definitely loop to a initial state.

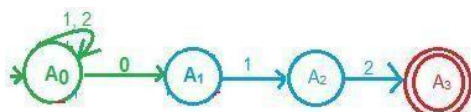


Figure 1.22: conversion of NFA state A_0 to DFA state.

From this transition we come to know that the termination state for the symbols '1', '2' is state A_0 . From this transition we come to know that the termination state for the symbol '0' is state A_1 .

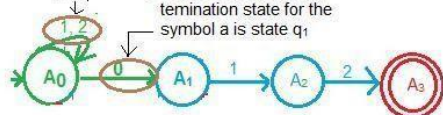


Figure 1.23: A_0 is termination for symbol 1, 2 and A_1 is termination for symbol 0

Termination state for symbol '0' is A_1 and for '1' and '2' is A_0 . Outgoing transition from A_1 on symbol '0' is loop and for '2' is A_0 . From A_2 on symbol '0' to A_1 and on symbol '1' to A_0 . From A_3 on symbol '0' to A_1 and on symbols '1' and '2' to A_0 .

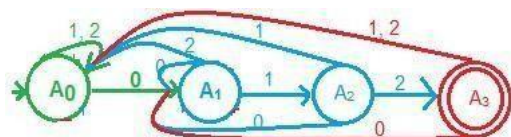


Figure 1.24: DFA which accepts all the strings ending with 012. We

can define above DFA as $M_{DFA} = (A, \Sigma, \delta, IS, \{FS\})$

$A = \{A_0, A_1, A_2, A_3\}; \Sigma = \{0, 1, 2\}$

δ :

$\delta(A_0, 0) = A_1$ $\delta(A_0, 1) = A_0$ $\delta(A_0, 2) = A_0$

$\delta(A_1, 0) = A_1$ $\delta(A_1, 1) = A_2$ $\delta(A_1, 2) = A_0$

$\delta(A_2, 0) = A_1$ $\delta(A_2, 1) = A_0$ $\delta(A_2, 2) = A_3$

$\delta(A_3, 0) = A_1$ $\delta(A_3, 1) = A_0$ $\delta(A_3, 2) = A_0$

$IS = A_0$; $FS = A_3$

Example 1.10: Design a DFA which accepts all the strings beginning with symbol 'a' over the alphabet

$\Sigma = \{a, b\}$.

$L = \{a, aa, ab, abb, aba, aaa, \dots\}$

- Draw the NFA for the smallest string of the language. The NFA for the string 'a' is

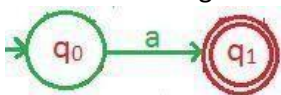


Figure 1.25: NFA for the string 'a'.

Convert all states of NFA to DFA, that is from each state two outgoing transitions, one for symbol 'a' another for 'b'. From initial state only one outgoing transition on symbol 'a', one more outgoing transition is required that is on 'b'. If the condition in the given problem is, beginwith,theoutgoingtransitionon theother symbol 'b',istonestate calleddeadstate. **Dead state**: In finite automata the string travers from initial state to final state, in some situation we make finite automata to die called dead state. Dead state is a non-final state, from which all outgoing transitions are loop to the same state.

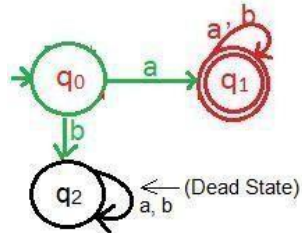


Figure 1.26: DFA which accepts all the strings beginning with 'a'.

- ii. Condition is begin, outgoing transitions from final state is loop.

We can define above DFA as $M_{DFA} = (Q, \Sigma, \delta, IS, \{FS\})$ $Q = \{$

$q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

$\delta(q_0, a) = q_1, \delta(q_0, b) = q_2$ δ

$(q_1, a) = q_1, \delta(q_1, b) = q_1$ δ

$(q_2, a) = q_2, \delta(q_2, b) = q_2$ IS

$= q_0, FS = \{q_1\}$

Example 1.11: Design a DFA which accepts all the strings begins with substring 01 over the alphabet

$\Sigma = \{0, 1\}$.

$L = \{01, 010, 011, 0100, 0101, \dots\}$.

- i. Draw the NFA for the smallest string 01 of the language. The NFA for the string 01 is

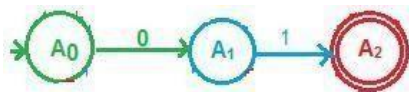


Figure 1.27: NFA for the string 01.

- ii. Convert all states of NFA to DFA, that is from each state two outgoing transitions, one for symbol '0' another for '1'. From initial and intermediate states only one outgoing transition, one more outgoing transition is required. If the condition in the given problem is, begin with, the outgoing transition from both the states to new state called dead state.

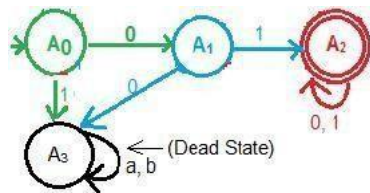


Figure 1.28: DFA which accepts all the string begin with 01.

iii. Condition is begin, outgoing transitions from final state A_2 on symbol '0' and '1' is loop. We can define above DFA as $M_{DFA} = (A, \Sigma, \delta, IS, \{FS\})$

$A = \{A_0, A_1, A_2, A_3\}; \Sigma = \{0, 1\}$

δ :

$\delta(A_0, 0) = A_1$ $\delta(A_0, 1) = A_3$

$\delta(A_1, 0) = A_3$

$\delta(A_1, 1) = A_2$

$\delta(A_2, 0) = A_2$ $\delta(A_2, 1) = A_2$

$\delta(A_3, 0) = A_3$ $\delta(A_3, 1) = A_3$

$FS = A_2$

Example 1.12: Design a DFA which accept all the strings begins with 01 or ends with 01 or both over the alphabet $\Sigma = \{0, 1\}$.

$L = \{01, 010, 011, 0100, 001, 101, 0001, 1001, 0101, 01101, \dots\}$

We have already designed both the DFAs, joining of these DFAs, is the solution for the given problem.

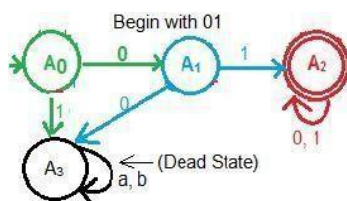


Figure 1.29: DFA which accepts all the string begin with 01.

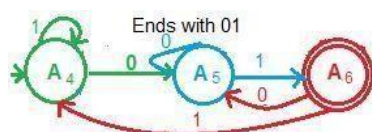


Figure 1.30: DFA which accepts all the string ends with 01.

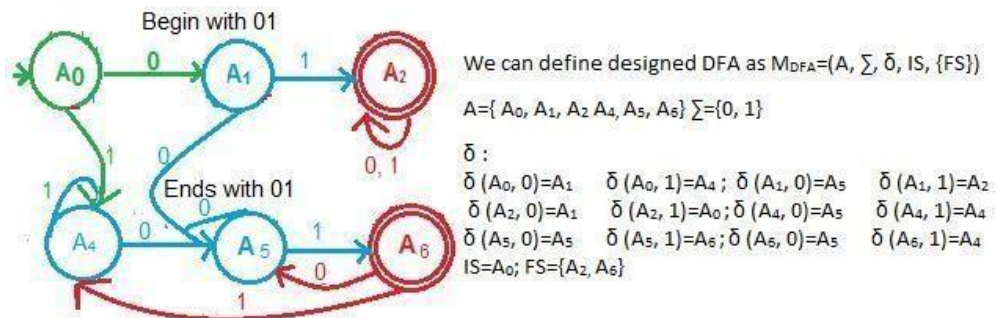


Figure 1.31: DFA which accepts all the strings begin with 01, end with 01, and end with 01.

Example 1.13: Design a DFA which accepts all the binary strings divisible by 2. $L = \{ \epsilon,$

$10, 100, 110, \dots \}$

To design DFA we need to make the following assumptions.

$q_0 \rightarrow 0 \rightarrow q_0$
 $q_0 \rightarrow 1 \rightarrow q_1$
 $q_0 \rightarrow 10 \rightarrow q_0$
 $q_0 \rightarrow 11 \rightarrow q_1$
 $q_0 \rightarrow 100 \rightarrow q_0$
 $q_0 \rightarrow 101 \rightarrow q_1$

From the above assumption, we can draw the following DFA.

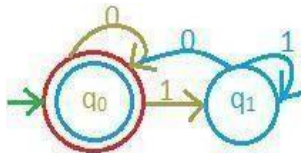


Figure 1.32: DFA which accepts all the strings divisible by 2. We can

define above DFA as $M_{DFA} = (Q, \Sigma, \delta, IS, \{FS\})$

$Q = \{q_0, q_1\}$

$\Sigma = \{0, 1\}$

$\delta(q_0, 0) = q_0$ $\delta(q_0, 1) = q_1$

$\delta(q_1, 0) = q_0$ $\delta(q_1, 1) = q_1$

$IS = q_0$ $FS = \{q_0\}$

Example:

Example 1.14: Design a DFA which accepts all the binary strings divisible by 5. $L = \{ \epsilon,$

$101, 1010, 1111, \dots \}$

To design DFA we need to make the following assumptions.

$q_0 \rightarrow 0 \rightarrow q_0$ $q_0 \rightarrow 101 \rightarrow q_0$
 $q_0 \rightarrow 1 \rightarrow q_1$ $q_0 \rightarrow 110 \rightarrow q_1$
 $q_0 \rightarrow 10 \rightarrow q_2$ $q_0 \rightarrow 111 \rightarrow q_2$
 $q_0 \rightarrow 100 \rightarrow q_3$ $q_0 \rightarrow 1000 \rightarrow q_3$
 $q_0 \rightarrow 11 \rightarrow q_4$ $q_0 \rightarrow 1001 \rightarrow q_4$
 $q_0 \rightarrow 100 \rightarrow q_4$ $q_0 \rightarrow 1010 \rightarrow q_5$

According to assumption we can draw the following DFA

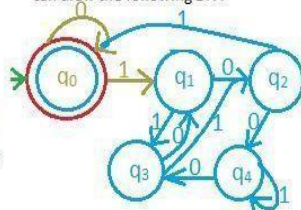


Figure 1.33: DFA which accepts all the strings divisible by 5.

We can define, designed DFA as $M_{DFA} = (A, \Sigma, \delta, IS, \{FS\})$ $A = \{$

$q_0, q_1, q_2, q_3, q_4\}$ $\Sigma = \{0, 1\}$

δ :

$\delta(q_0, 0) = q_0$ $\delta(q_0, 1) = q_1$; $\delta(q_1, 0) = q_2$ $\delta(q_1, 1) = q_3$

$\delta(q_2, 0) = q_4$ $\delta(q_2, 1) = q_0$; $\delta(q_3, 0) = q_1$ $\delta(q_3, 1) = q_2$

$\delta(q_4, 0) = q_3$ $\delta(q_4, 1) = q_4$;

$IS = q_0$; $FS = q_0$

Example 1.15: Design a DFA which accept string over x, y every block of length 3 contains at least one x . To design a DFA we need to make following assumptions,

1. If there is no 'x' in the string, such a string should not be accepted.

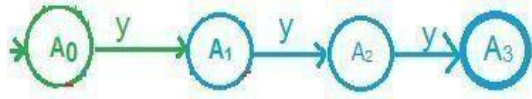


Figure 1.34: NFA which is not accepting three y's string.

2. One 'x' in the string, accepted by finite automata.

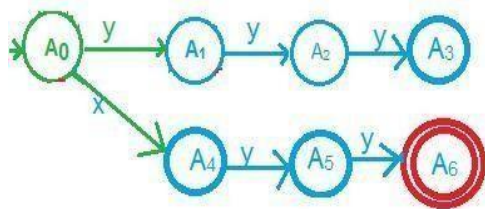


Figure 1.35: NFA which is accepting three symbol string in which one symbol is x.

3. The position of A_1 and A_5 is same as A_0 and A_4 . In between A_0 and A_4 the symbol is 'x', in between A_1 and A_5 is also x and so on.

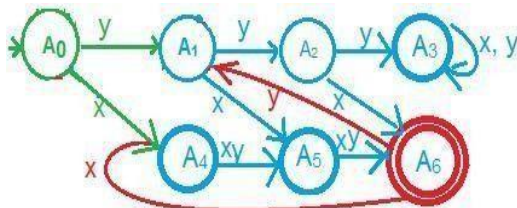


Figure 1.36: DFA which accept string over x, y if every block of length 3 contains at least one x .

We can define designed DFA as $M_{DFA} = (A, \Sigma, \delta, IS, \{FS\})$ $A = \{$

$A_0, A_1, A_2, A_4, A_5, A_6\}$ $\Sigma = \{x, y\}$

δ :

$\delta(A_0, x) = A_4$ $\delta(A_0, y) = A_1$; $\delta(A_1, x) = A_5$ $\delta(A_1, y) = A_2$

$\delta(A_2, x) = A_6$ $\delta(A_2, y) = A_3$; $\delta(A_3, x) = A_3$ $\delta(A_3, y) = A_3$

$\delta(A_4, x) = A_5$ $\delta(A_4, y) = A_5$; $\delta(A_5, x) = A_6$ $\delta(A_5, y) = A_6$;

$$\delta(A_6, x) = A_4$$

$$\delta(A_6, y) = A_1$$

$$S = A_0; FS = \{A_6\}$$

Example 1.16: Design a DFA which accepts even number of r 's and even number of s 's over the symbol $\Sigma = \{r, s\}$.

- i. Draw separate DFA, one accept even number of r 's and another one accept even number of s 's.

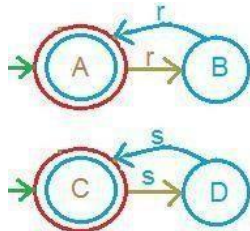


Figure 1.37: DFA's accept even symbols string over r, s .

- ii. Join two DFA, we get four new states AB, AC, BC, BD. Designed DFA accept even number of r and even number of s , out of four states AC satisfied this condition, hence this state is a final state.

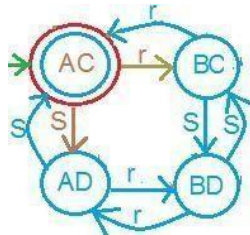


Figure 1.38: DFA which accept even number of r and s .

We can define designed DFA as $M_{DFA} = (State, \Sigma, \delta, IS, \{FS\})$

$$State = \{AC, AD, BC, BD\} \quad \Sigma = \{r, s\}$$

δ :

$$\begin{array}{llll} \delta(AC, r) = BC & \delta(AC, s) = AD & \delta(BC, r) = AC & \delta(BC, s) = BD \\ \delta(AD, r) = BD & \delta(AD, s) = AC & \delta(BD, r) = AD & \delta(BD, s) = BC \end{array}$$

$$IS = AC; FS = \{AC\}$$

Example 1.17: Design a DFA which accepts all the strings described by the language $L = \{N_0(w) \geq 1 \text{ and } N_1(w) = 2\}$

$$L = \{011, 110, 0011, 1010, 11100, 10101, \dots\}$$

- i. Exchange 1 symbol of first string with the last symbol of the first string, the string obtained is the second string of the language.
- ii. Draw NFA for two smallest strings, for both NFA initial and final states are same.

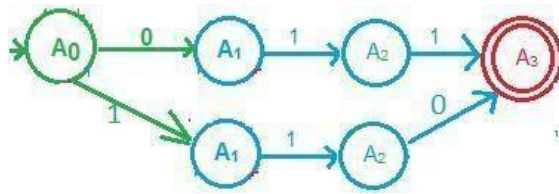


Figure1.39:TwoDFAsforthestring011and110withsameinitialandfinal state.

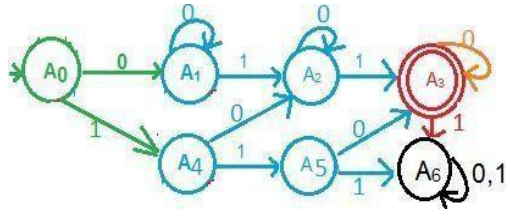


Figure1.40:DFAwhichacceptallthestringsdescribedbythelanguage $L=\{N_0(w) \geq 1 \text{ and } N_1(w) = 2\}$. We can

define designed DFA as $M_{DFA} = (A, \Sigma, \delta, IS, \{FS\})$

$A = \{A_0, A_1, A_2, A_3, A_4, A_5, A_6\}$ $\Sigma = \{0, 1\}$

δ :

$\delta(A_0, 0) = A_1$ $\delta(A_0, 1) = A_4$; $\delta(A_1, 0) = A_0$ $\delta(A_1, 1) = A_2$

$\delta(A_2, 0) = A_4$ $\delta(A_2, 1) = A_3$; $\delta(A_4, 0) = A_2$ $\delta(A_4, 1) = A_5$

$\delta(A_5, 0) = A_3$ $\delta(A_5, 1) = A_6$; $\delta(A_6, 0) = A_6$ $\delta(A_6, 1) = A_6$

$IS = A_0$; $FS = \{A_3\}$

NondeterministicFiniteAutomata(NFA)

In DFA the outgoing transitions are defined in advance, where as in NFA the outgoing transitions are not defined in advance. In DFA, the string is either accepted or not accepted, where as in NFA both are possible. The finite automata shown in the figure 1.41 is a NFA. There is no outgoing transition for the symbol 'a' from the state q_1 .

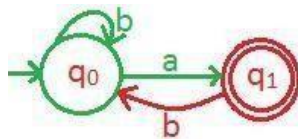


Figure1.41:NondeterministicFiniteAutomata

We can define above NFA as $M_{DFA} = (Q, \Sigma, \delta, IS, \{FS\})$ $Q = \{$

$q_0, q_1, \}$

$\Sigma = \{a, b\}$

δ :

$Q^* \Sigma \rightarrow 2^Q$

$\delta(q_0, a) = q_1$ $\delta(q_0, b) = q_0$ $\delta(q_1,$

$b) = q_0$

$\delta(q_2, a) = q_2$ $\delta(q_2, b) = q_2$

$IS = q_0$ $FS = \{q_1\}$

The NFA shown in figure 1.42 is, accepting as well as not accepting the string 'hod'.

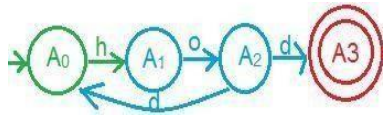


Figure 1.42: Non-deterministic finite automata

Example 1.18: Design NFA which accept all strings, if second symbol from right side is 1 over the alphabet $\Sigma = \{0, 1\}$.

$L = \{10, 110, 010, 0110, \dots\}$

- i. First step draw the final state.



Figure 1.43: Final state

- ii. Draw the intermediate state and transition for the symbols '0' and '1'.

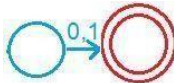


Figure 1.44: Intermediate and final states

- iii. Draw the initial state and transition for the symbol '1'.

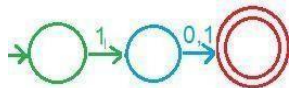


Figure 1.45: Non-deterministic finite automata accepting two strings 10, 11.

- iv. Designed NFA has to accept all the strings, if second symbol from right side is symbol '1', to do this, mark loop to initial state on symbols '0' and '1'.

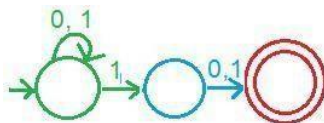


Figure 1.46: NFA which accepts all strings, if second symbol from right side is 1 over the alphabet $\Sigma = \{0, 1\}$.

- v. Name the states and stop.

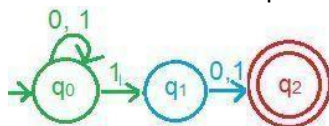


Figure 1.47: NFA which accepts all strings, if second symbol from right side is 1 over the alphabet $\Sigma = \{0, 1\}$.

Definition of NFA: NFA can be defined as

$$M_{DFA} = (Q, \Sigma, \delta, IS, \{FS\})$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

δ :

$$\delta(q_0, 0) = q_0, \delta(q_0, 1) = \{q_0, q_1\}$$

$$\delta(q_1, 0) = q_2$$

$$\delta(q_1, 1) = q_2$$

$$IS = q_0, FS = \{q_2\}$$

Equivalence of NFA and DFA:

We have already seen both NFA and DFA, both are same in the definition but they differ in transition function. If number of states in NFA has two states q_0 and q_1 . Here, q_0 is initial state and q_1 is final state. The equivalent states in DFA's are q_0 , q_1 and $[q_0, q_1]$. Where q_0 is initial state and (q_1) , (q_0, q_1) are the final states. The pair (q_0, q_1) is final state because; one of its states is final state. They are represented as shown in figure 1.48

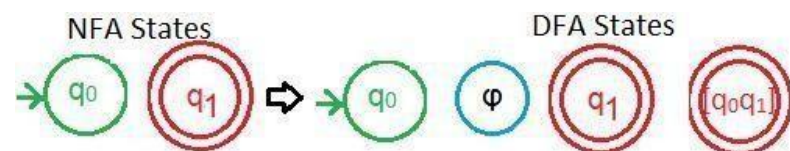


Figure 1.48: Equivalence states of NFA and DFA.

Example 1.19: Convert the NFA shown in figure 1.49 into its equivalent DFA.

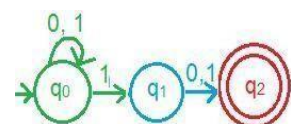


Figure 1.49: Non-deterministic finite automata of Example 1.19.

The conversion begins with the initial state of the NFA which is also an initial state of DFA. Conversion generates number of new states, for every new state we are finding the outgoing transitions for different symbols.

Table 1.4: Different tables for the conversion of NFA to DFA.

$q \backslash \Sigma$	0	1
$\rightarrow q_0$	q_0	$[q_0 q_1]$ — New state
q_0	q_0	$[q_0 q_1]$ — New state
$[q_0 q_1]$	$[q_0 q_2]$	$[q_0 q_1 q_2]$

New states

$$\delta([q_0 q_1], 0) = \delta(q_0, 0) \cup \delta(q_1, 0)$$

$$= q_0 \cup q_2$$

$$= q_0 q_2$$

$$\delta([q_0 q_1], 1) = \delta(q_0, 1) \cup \delta(q_1, 1)$$

$$= q_0 q_1 \cup q_2$$

$$= q_0 q_1 q_2$$

$Q \backslash \Sigma$	0	1
$\rightarrow q_0$	q_0	$[q_0 q_1]$
$[q_0 q_1]$	$q_0 q_2$	$q_0 q_1 q_2$
$q_0 q_2$	q_0	$[q_0 q_1]$

$$\begin{aligned}\delta([q_0 q_2], 0) &= \delta(q_0, 0) \cup \delta(q_2, 0) \\ &= q_0 \cup \emptyset \\ &= q_0 \\ \delta([q_0 q_2], 1) &= \delta(q_0, 1) \cup \delta(q_2, 1) \\ &= q_0 q_1 \cup \emptyset \\ &= q_0 q_1\end{aligned}$$

$Q \backslash \Sigma$	0	1
$\rightarrow q_0$	q_0	$[q_0 q_1]$
$[q_0 q_1]$	$q_0 q_2$	$q_0 q_1 q_2$
$[q_0 q_2]$	q_0	$[q_0 q_1]$
$[q_0 q_1 q_2]$	$[q_0 q_2]$	$[q_0 q_1 q_2]$

$$\begin{aligned}\delta([q_0 q_1 q_2], 0) &= \delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0) \\ &= q_0 \cup q_2 \cup \emptyset \\ &= q_0 q_2 \\ \delta([q_0 q_1 q_2], 1) &= \delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1) \\ &= q_0 q_1 \cup q_2 \cup \emptyset \\ &= q_0 q_1 q_2\end{aligned}$$

No further new states; mark the final state of DFA. Any pair of state which consists of final state of the NFA, the particular pair is a final state in DFA.

$Q \backslash \Sigma$	0	1
$\rightarrow q_0$	q_0	$[q_0 q_1]$
$[q_0 q_1]$	$q_0 q_2$	$q_0 q_1 q_2$
$*[q_0 q_2]$	q_0	$[q_0 q_1]$
$*[q_0 q_1 q_2]$	$[q_0 q_2]$	$[q_0 q_1 q_2]$

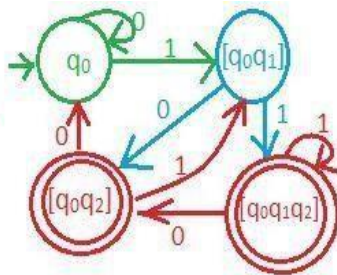


Figure1.50:EquivalenceDFAoffigure1.49

Application of Finite automata:

Finite automata is used to search a text. There are two methods used to search text, they are:

- Nondeterministic Finite Automata (NFA)
- Deterministic Finite Automata (DFA)

To search a text "shine" and "hire" the NFA shown in figure 1.51 is used. The

input alphabets are $\Sigma = \{s, h, i, n, e, r\}$

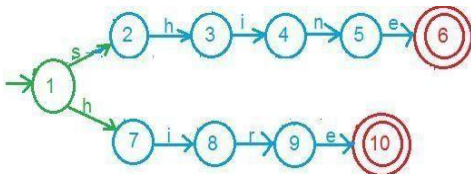


Figure1.51:NFA used to search strings shine and hire

To search a text "shine" and "hire" the DFA shown in figure 1.52 is used

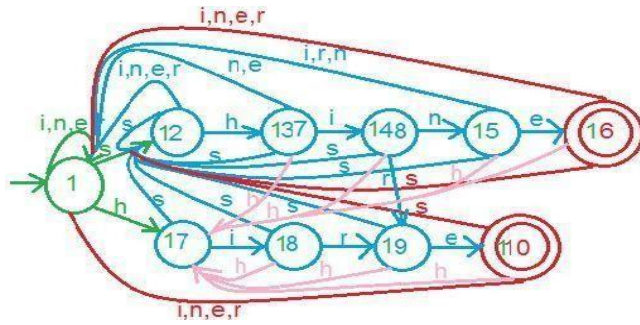


Figure 1.52: DFA used to search strings ending with shine and hire

Example 1.20: Design a finite automata, that reads string made up of letters HONDA, and recognize those string that contain the word HOD as a substring.

- Find out the alphabet from the string HONDA. The input symbols are $\Sigma = \{H, O, N, D, A\}$
- Draw the NFA for the substring HOD.

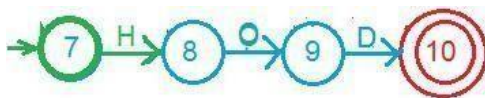


Figure 1.53: NFA used to search string HOD

- Convert NFA state of figure 1.53 into DFA states.

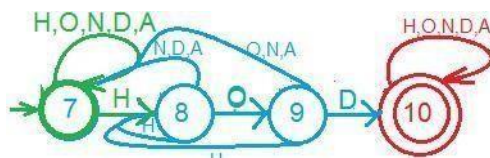


Figure 1.54: DFA accepts all the strings end with HOD over the symbols $\{H, O, N, D, A\}$

NFA with ϵ -transition:

Till now whatever automata we studied were characterized by a transition on the symbols. But there are some finite automata having transition for empty string also, such NFAs are called NFA- ϵ transition.

ϵ -closure(q_0): states which are closed/related to state q_0 on empty transition (ϵ) is called ϵ -closure(q_0).

In DFA and NFA $\hat{\delta}(q_0, \epsilon) = q_0$, whereas in ϵ -NFA

- $\hat{\delta}(q, \epsilon) = \epsilon\text{-closure}(\hat{\delta}(q, \epsilon)) = \epsilon\text{-closure}(q)$.
- $\hat{\delta}(q, x) = \epsilon\text{-closure}(\delta(\hat{\delta}(q, \epsilon), x)) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q), x))$.
- $\hat{\delta}(q, wx) = \epsilon\text{-closure}(\delta(\hat{\delta}(q, w), x))$.

Example 1.21: Find out ϵ -closure of all the states of ϵ -NFA shown in figure 1.55.

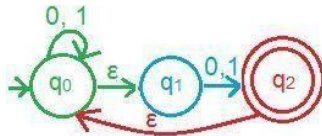


Figure 1.55: NFA with ϵ -transition of Example 1.21.

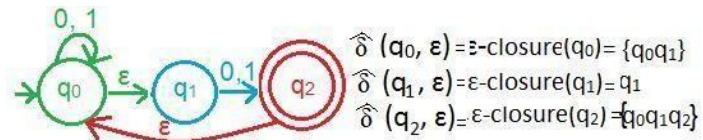


Figure 1.56: ϵ -closure of all the states of figure 1.55

Example 1.22: Convert the ϵ -NFA shown in figure 1.57 into NFA

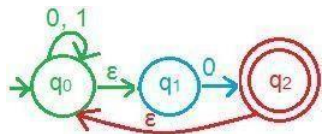


Figure 1.57: ϵ -NFA of Example 1.22.

First find out the ϵ -closure of all the states.

Note: state ϵ -closure consists of final state of ϵ -NFA, the particular state is a final state in NFA.

$$\hat{\delta}(q_0, \epsilon) = \epsilon\text{-closure}(q_0) = \{q_0, q_1\}$$

$$\hat{\delta}(q_1, \epsilon) = \epsilon\text{-closure}(q_1) = q_1$$

$$\hat{\delta}(q_2, \epsilon) = \epsilon\text{-closure}(q_2) = \{q_0, q_1, q_2\}$$

$$\hat{\delta}(q_0, 0) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_0, \epsilon), 0)).$$

$$= \epsilon\text{-closure}(\delta(\{q_0, q_1\}, 0)).$$

$$= \epsilon\text{-closure}(\delta(q_0, 0) \cup \delta(q_1, 0)).$$

$$= \epsilon\text{-closure}(q_0 \cup q_2).$$

$$= \epsilon\text{-closure}(q_0) \cup \epsilon\text{-closure}(q_2).$$

$$= \{q_0, q_1\} \cup \{q_0, q_1, q_2\}$$

$$= \{q_0, q_1, q_2\}$$

$$\hat{\delta}(q_0, 1) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_0, \epsilon), 1)).$$

$$= \epsilon\text{-closure}(\delta(\{q_0, q_1\}, 1)).$$

$$= \epsilon\text{-closure}(\delta(q_0, 1) \cup \delta(q_1, 1)).$$

$$=\epsilon\text{-closure}(q_0 \cup \phi).$$

$$=\epsilon\text{-closure}(q_0) \cup \epsilon\text{-closure}(\phi).$$

$$=\{q_0 q_1\}$$

$$\hat{\delta}(q_1, 0) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_1, \epsilon), 0)).$$

$$=\epsilon\text{-closure}(\delta(q_1, 0)).$$

$$=\epsilon\text{-closure}(q_2).$$

$$=\{q_0 q_1 q_2\}$$

$$\hat{\delta}(q_1, 1) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_1, \epsilon), 1)).$$

$$=\epsilon\text{-closure}(\delta(q_1, 1)).$$

$$=\epsilon\text{-closure}(\phi).$$

$$=\phi$$

$$\hat{\delta}(q_2, 0) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_2, \epsilon), 0)).$$

$$=\epsilon\text{-closure}(\delta(\{q_0 q_1 q_2\}, 0)).$$

$$=\epsilon\text{-closure}(\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0)).$$

$$=\epsilon\text{-closure}(q_0 \cup q_2 \cup \phi).$$

$$=\epsilon\text{-closure}(q_0) \cup \epsilon\text{-closure}(q_2) \cup \epsilon\text{-closure}(\phi).$$

$$=\{q_0 q_1\} \cup \{q_0 q_1 q_2\} \cup (\phi)$$

$$=\{q_0 q_1 q_2\}$$

$$\hat{\delta}(q_2, 1) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_2, \epsilon), 1)).$$

$$=\epsilon\text{-closure}(\delta(\{q_0 q_1 q_2\}, 1)).$$

$$=\epsilon\text{-closure}(\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1)).$$

$$=\epsilon\text{-closure}(q_0 \cup \phi \cup \phi).$$

$$=\epsilon\text{-closure}(q_0) \cup \epsilon\text{-closure}(\phi) \cup \epsilon\text{-closure}(\phi)$$

$$=(q_0 q_1) \cup (\phi) \cup (\phi)$$

$$=\{q_0 q_1\}$$

Table 1.5: Equivalence table of ϵ -NFA and NFA.

$Q \backslash \Sigma$	0	1
q_0	$\{q_0 q_1 q_2\}$	$\{q_0 q_1\}$
q_1	$\{q_0 q_1 q_2\}$	$\emptyset$
q_2	$\{q_0 q_1 q_2\}$	$\{q_0 q_1\}$

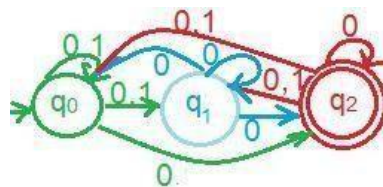


Figure 1.58: NFA without ϵ -transition.

Conversion of ϵ -NFA to DFA:

It is same as conversion of NFA to DFA, begins with initial state of ϵ -NFA.

Example 1.23: Convert the ϵ -NFA shown in figure 1.59 into DFA.

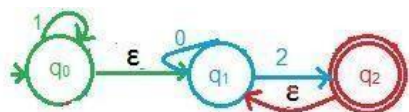


Figure 1.59: ϵ -NFA of Example 1.23.

First find out the ϵ -closure of all the states

$$\hat{\delta}(q_0, \epsilon) = \epsilon\text{-closure}(q_0) = \{q_0 q_1\}$$

$$\hat{\delta}(q_1, \epsilon) = \epsilon\text{-closure}(q_1) = q_1$$

$$\hat{\delta}(q_2, \epsilon) = \epsilon\text{-closure}(q_2) = \{q_1 q_2\}$$

Table 1.6: Equivalence table of ϵ -NFA and NFA.

$Q \backslash \Sigma$	0	1	2
q_0	q_1	$\{q_0 q_1\}$	$\{q_1 q_2\}$
q_1	q_1	$\emptyset$	$\{q_1 q_2\}$
$\{q_0 q_1\}$	q_1	$\{q_0 q_1\}$	$\{q_1 q_2\}$
$\{q_1 q_2\}$	q_1	$\emptyset$	$\{q_1 q_2\}$

$$\hat{\delta}(q_0, 0) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_0, \epsilon), 0)).$$

$$= \epsilon\text{-closure}(\delta(\{q_0 q_1\}, 0)).$$

$$=\epsilon\text{-closure}(\delta(q_0,0)\cup\delta(q_1,0)).$$

$$=\epsilon\text{-closure}(\phi\cup q_1).$$

$$=\epsilon\text{-closure}(\phi)\cup\epsilon\text{-closure}(q_1).$$

$$=\phi\cup q_1$$

$$=q_1$$

$$\hat{\delta}(q_0,1)=\epsilon\text{-closure}(\delta(\hat{\delta}(q_0,\epsilon),1).$$

$$=\epsilon\text{-closure}(\delta(\{q_0q_1\},1).$$

$$=\epsilon\text{-closure}(\delta(q_0,1)\cup\delta(q_1,1)).$$

$$=\epsilon\text{-closure}(q_0\cup\phi).$$

$$=\epsilon\text{-closure}(q_0)\cup\epsilon\text{-closure}(\phi).$$

$$=\{q_0q_1\}\cup\phi$$

$$=[q_0q_1]$$

$$\hat{\delta}(q_0,2)=\epsilon\text{-closure}(\delta(\hat{\delta}(q_0,\epsilon),2).$$

$$=\epsilon\text{-closure}(\delta(\{q_0q_1\},2).$$

$$=\epsilon\text{-closure}(\delta(q_0,2)\cup\delta(q_1,2)).$$

$$=\epsilon\text{-closure}(\phi\cup q_2).$$

$$=\epsilon\text{-closure}(q_2).$$

$$=[q_1q_2]$$

$$\hat{\delta}(q_1,0)=\epsilon\text{-closure}(\delta(\hat{\delta}(q_1,\epsilon),0).$$

$$=\epsilon\text{-closure}(\delta(q_1,0).$$

$$=\epsilon\text{-closure}(q_1).$$

$$=q_1$$

$$\hat{\delta}(q_1,1)=\epsilon\text{-closure}(\delta(\hat{\delta}(q_1,\epsilon),1).$$

$$=\epsilon\text{-closure}(\delta(q_1,1).$$

$$=\epsilon\text{-closure}(\phi).$$

$$=\phi$$

$$\widehat{\delta}(q_1, 2) = \varepsilon\text{-closure}(\delta(\widehat{\delta}(q_1, \varepsilon), 2)).$$

$$= \varepsilon\text{-closure}(\delta(q_1, 2))$$

$$= \varepsilon\text{-closure}(q_2)$$

$$=[q_1, q_2]$$

$$\widehat{\delta}([q_0, q_1], 0) = \varepsilon\text{-closure}(\delta(\widehat{\delta}([q_0, q_1], \varepsilon), 0))$$

$$= \varepsilon\text{-closure}(\delta(\widehat{\delta}(q_0, \varepsilon) \cup \widehat{\delta}(q_1, \varepsilon)), 0).$$

$$= \varepsilon\text{-closure}(\delta((q_0, q_1) \cup q_1), 0).$$

$$= \varepsilon\text{-closure}(\delta((q_0, q_1), 0)).$$

$$= \varepsilon\text{-closure}(\delta(q_0, 0) \cup \delta(q_1, 0)).$$

$$= \varepsilon\text{-closure}(\phi \cup q_1)$$

$$= \varepsilon\text{-closure}(q_1)$$

$$= q_1$$

$$\widehat{\delta}([q_0, q_1], 1) = \varepsilon\text{-closure}(\delta(\widehat{\delta}([q_0, q_1], \varepsilon), 1))$$

$$= \varepsilon\text{-closure}(\delta(\widehat{\delta}(q_0, \varepsilon) \cup \widehat{\delta}(q_1, \varepsilon)), 1).$$

$$= \varepsilon\text{-closure}(\delta((q_0, q_1) \cup q_1), 1).$$

$$= \varepsilon\text{-closure}(\delta((q_0, q_1), 1)).$$

$$= \varepsilon\text{-closure}(\delta(q_0, 1) \cup \delta(q_1, 1)).$$

$$= \varepsilon\text{-closure}(q_0 \cup \phi)$$

$$= \varepsilon\text{-closure}(q_0)$$

$$=[q_0, q_1]$$

$$\widehat{\delta}([q_0, q_1], 2) = \varepsilon\text{-closure}(\delta(\widehat{\delta}([q_0, q_1], \varepsilon), 2))$$

$$= \varepsilon\text{-closure}(\delta(\widehat{\delta}(q_0, \varepsilon) \cup \widehat{\delta}(q_1, \varepsilon)), 2).$$

$$= \varepsilon\text{-closure}(\delta((q_0, q_1) \cup q_1), 2).$$

$$= \varepsilon\text{-closure}(\delta((q_0, q_1), 2)).$$

$$= \varepsilon\text{-closure}(\delta(q_0, 2) \cup \delta(q_1, 2)).$$

$$=\epsilon\text{-closure}(\phi \cup q_2)$$

$$=\epsilon\text{-closure}(q_2)$$

$$=[q_1q_2]$$

$$\hat{\delta}([q_1q_2], 0) = \epsilon\text{-closure}(\delta(\hat{\delta}([q_1q_2], \epsilon), 0))$$

$$=\epsilon\text{-closure}(\delta(\hat{\delta}(q_1, \epsilon) \cup \hat{\delta}(q_2, \epsilon), 0)).$$

$$=[q_1q_2]$$

Table 1.7: Equivalence table of ϵ -NFA and DFA.

$Q \backslash \Sigma$	0	1	2
q_0	q_1	$[q_0q_1]$	$[q_1q_2]$
q_1	q_1	ϕ	$[q_1q_2]$
$[q_0q_1]$	q_1	$[q_0q_1]$	$[q_1q_2]$
$[q_1q_2]$	q_1	ϕ	$[q_1q_2]$
ϕ	ϕ	ϕ	ϕ

$$\hat{\delta}(\phi, (0, 1, 2)) = \phi$$

Mark the final states, any pair which has a final state of ϵ -NFA the particular pair is a final state in DFA.

Table 1.7: Equivalence table of ϵ -NFA and DFA .

$Q \backslash \Sigma$	0	1	2
q_0	q_1	$[q_0q_1]$	$[q_1q_2]$
q_1	q_1	ϕ	$[q_1q_2]$
$[q_0q_1]$	q_1	$[q_0q_1]$	$[q_1q_2]$
$[q_1q_2]$	q_1	ϕ	$[q_1q_2]$
ϕ	ϕ	ϕ	ϕ

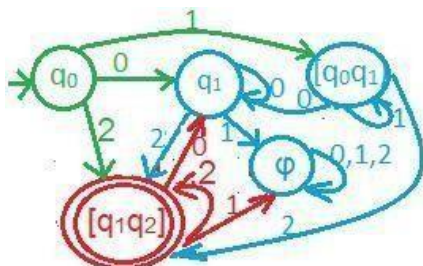


Figure 1.60: DFA of figure 1.59.